

DNS Threat Hunting with Zeek and Splunk

Detecting DNS Tunneling and C2 Beaconing
in Live, Self-Generated Network Traffic

Overview

I built this project to practice real DNS threat hunting — not just reading log files someone else already collected, but capturing my own live network traffic and hunting through it the way a SOC analyst would. It runs on top of a real-time monitoring pipeline I built first: Zeek watching my network traffic, a Splunk Universal Forwarder shipping Zeek's logs over, and Splunk Enterprise doing the analysis.

In this project I used that pipeline to generate and investigate two attack techniques that abuse DNS: **DNS tunneling** and **C2 beaconing**. Both are explained in plain terms in the sections below, since the names alone don't tell you much. Each finding is also mapped to the **MITRE ATT&CK framework** — the industry-standard catalog of real-world attacker tactics and techniques — so the findings tie back to language any SOC team already uses.

Why “Threat Hunting” and Not “Log Analysis”

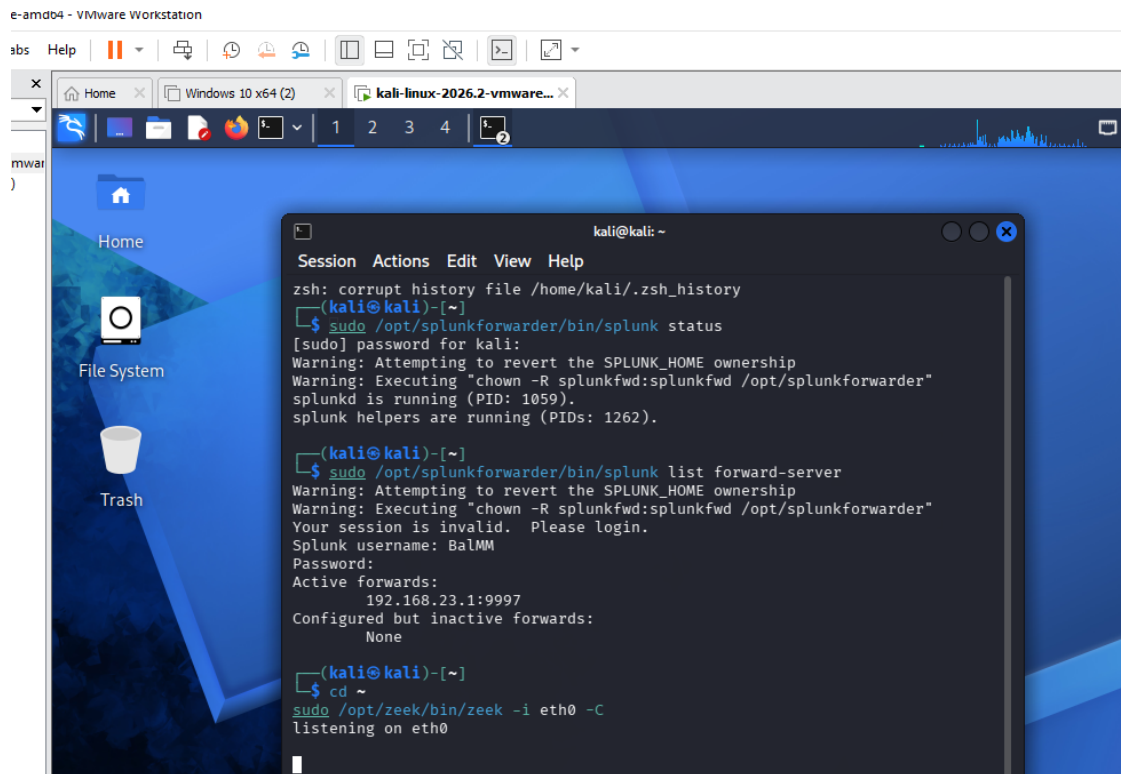
These sound similar but aren't the same skill. Log analysis is passive — you look at what happened after the fact. Threat hunting is proactive: you start with a hypothesis (“if tunneling were happening, what would it look like in my data?”), then build a search specifically designed to prove or disprove that hypothesis, before you know whether anything bad is actually there. That's the approach I used here — I picked two real attack techniques, simulated what they'd look like in DNS traffic, and then hunted for the patterns.

Lab Setup

Kali Linux VM running Zeek (captures live network traffic) and a Splunk Universal Forwarder (ships Zeek's logs out), sending everything to Splunk Enterprise on my host laptop over VMware's NAT network.

Component	Role	Details
Host laptop	Splunk Enterprise (SIEM)	Receives forwarded logs on TCP 9997
Kali Linux (VMware VM)	Sensor / traffic capture	Runs Zeek + Splunk Universal Forwarder
Network mode	VMware NAT	Shared 192.168.23.0/24 network via VMnet8

Since I'd built the pipeline the night before, I had to check everything was still alive before starting — the forwarder restarts itself automatically, but Zeek doesn't, so I had to start that fresh.



Confirming the Splunk Universal Forwarder was running and actively connected, before restarting Zeek's live packet capture.

How I Generated the Traffic

I generated three categories of DNS queries on purpose, so I'd have a realistic mix of normal and suspicious traffic to hunt through — a bit like hiding a needle in a haystack I built myself.

1. Normal baseline traffic

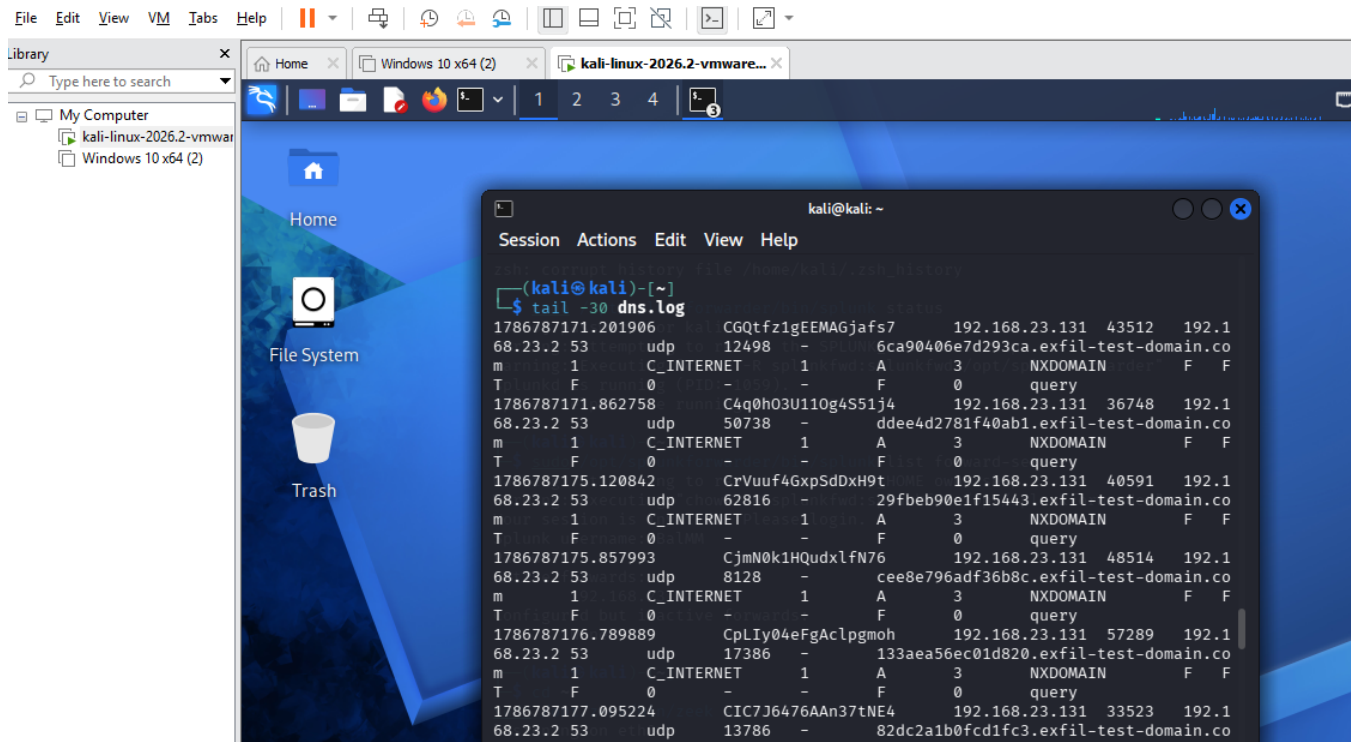
Everyday lookups to real, common sites — google.com, github.com, wikipedia.org, microsoft.com, amazonaws.com, netflix.com. This is my “what does normal look like” reference point.

2. Simulated DNS tunneling

DNS tunneling is a technique where an attacker sneaks stolen data out of a network by encoding it into DNS queries — specifically into the subdomain part of the domain name. It works because DNS traffic is almost never blocked by firewalls (every device needs it just to browse the internet), so it's a good hiding spot for smuggling data out. I simulated this by generating 20+ queries to random, 16-character hexadecimal (letters a–f and numbers 0–9) subdomains under one made-up domain, exfil-test-domain.com — mimicking what encoded stolen data would look like as a DNS query.

3. Simulated C2 beaconing

“C2” stands for command-and-control — the server an attacker uses to remotely control malware once it's on a victim's machine. Infected machines often “beacon” — send a small check-in query — to their C2 server on a regular timer, waiting for new instructions. I simulated this by sending a query to a made-up domain, suspicious-c2-domain.net, once every 60 seconds for 13 minutes, running quietly in the background.

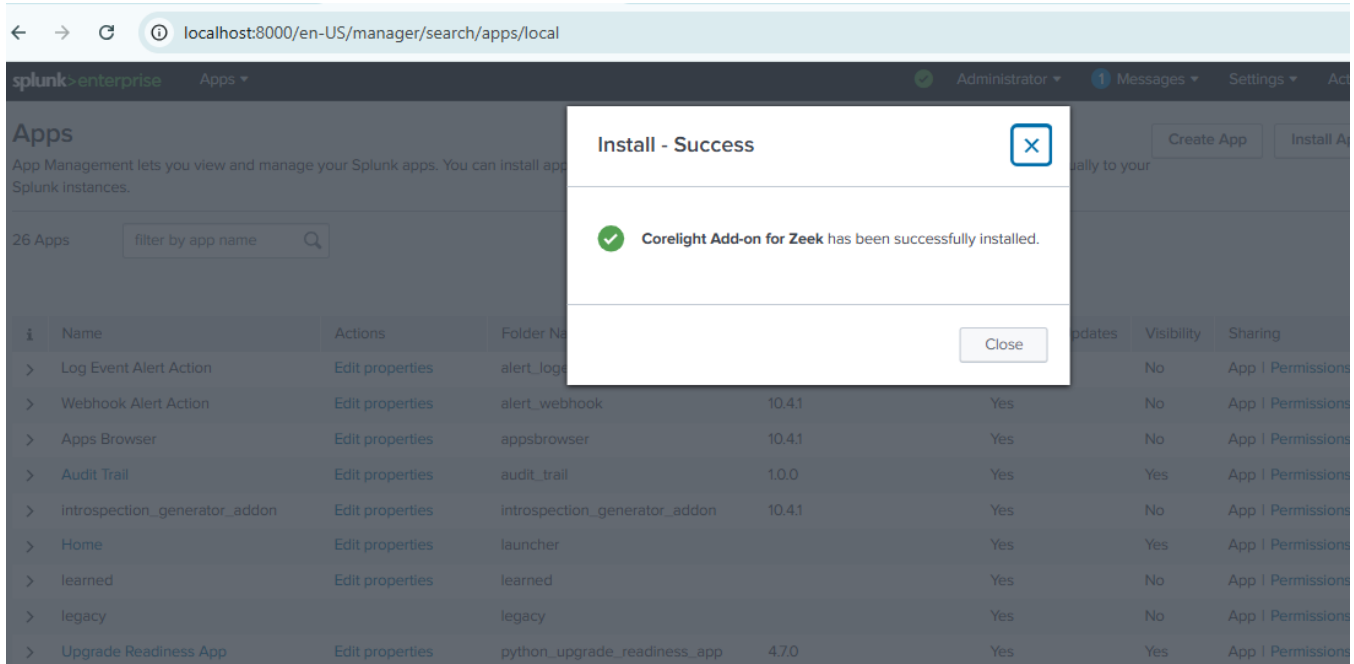


```
kali@kali: ~  
Session Actions Edit View Help  
$ tail -30 dns.log  
1786787171.201906 CGQtFz1gEEMAGjafs7 192.168.23.131 43512 192.1  
68.23.2 53 udp 12498 - 6ca90406e7d293ca.exfil-test-domain.co  
m 1 C_INTERNET 1 A 3 NXDOMAIN F F  
T F 0 - F 0 query  
1786787171.862758 C4q0h03U110g4S51j4 192.168.23.131 36748 192.1  
68.23.2 53 udp 50738 - ddee4d2781f40ab1.exfil-test-domain.co  
m 1 C_INTERNET 1 A 3 NXDOMAIN F F  
T F 0 - F 0 query  
1786787175.120842 CrVuuf4GxpSdDxH9t 192.168.23.131 40591 192.1  
68.23.2 53 udp 62816 - 29fbeb90e1f15443.exfil-test-domain.co  
m 1 C_INTERNET 1 A 3 NXDOMAIN F F  
T F 0 - F 0 query  
1786787175.857993 CjmN0k1HQudxlfN76 192.168.23.131 48514 192.1  
68.23.2 53 udp 8128 - cee8e796adf36b8c.exfil-test-domain.co  
m 1 C_INTERNET 1 A 3 NXDOMAIN F F  
T F 0 - F 0 query  
1786787176.789889 CpLIy04eFgAcLpgmoh 192.168.23.131 57289 192.1  
68.23.2 53 udp 17386 - 133aea56ec01d820.exfil-test-domain.co  
m 1 C_INTERNET 1 A 3 NXDOMAIN F F  
T F 0 - F 0 query  
1786787177.095224 CIC7J6476AAAn37tNE4 192.168.23.131 33523 192.1  
68.23.2 53 udp 13786 - 82dc2a1b0fcd1fc3.exfil-test-domain.co  
m 1 C_INTERNET 1 A 3 NXDOMAIN F F
```

Live dns.log output showing the simulated tunneling queries hitting exfil-test-domain.com, each returning NXDOMAIN (the domain doesn't actually exist — expected, since I made it up), captured live by Zeek.

A Real Problem I Ran Into: Splunk Wasn't Parsing the Data

Zeek writes its logs as TSV (tab-separated values) — basically a spreadsheet saved as plain text, where each column is separated by a tab character instead of a comma. To make sense of that in Splunk, I installed the **Corelight Add-on for Zeek**, a free add-on built specifically to teach Splunk how to split each Zeek log line into proper fields (like query, source IP, and the DNS response code).



The add-on installed and enabled inside Splunk Enterprise.

The problem: once I actually tested it, the add-on's parser was too strict. A chunk of my traffic — mostly background broadcast noise like mDNS/LLMNR (protocols devices use to find each other on a local network) — had fewer columns than Zeek's own header line said they should have. The add-on rejected those events outright instead of properly parsing them. Worse, even my clean, well-formed DNS query events came back with **zero extracted fields** — just one big blob of raw text.

How I fixed it

Instead of continuing to fight the add-on, I wrote my own field extraction directly in Splunk using the rex command. rex lets you use a regex (a pattern-matching syntax) to pull specific pieces out of a line of text and name them. Since I know Zeek separates every field with a literal tab character, I wrote a pattern that captures each tab-separated value and names it (source IP, destination IP, query, and so on) — no add-on required.

localhost:8000/en-US/app/search/search?q=search%20index%3Dmain%20host%3Dkali%20source%3D*dns.log%0A%7C%20eval%20fields%3Dsplit(_raw%2C%20...)

splunk> Enterprise

New Search

Save As

Create Table View

Close

Time range: Last 15 minutes

```
index=main host=kali source="*dns.log"
| eval fields=split(_raw, " ")
| eval ts=mindex(fields,0), uid=mindex(fields,1), src_ip=mindex(fields,2), src_port=mindex(fields,3), dest_ip=mindex(fields,4), dest_port=mindex(fields,5), proto=mindex(fields,6), query=mindex(fields,9), qtype_name=mindex(fields,13), rcode_name=mindex(fields,15)
| where NOT match(_raw, "^#")
| table _time, src_ip, dest_ip, query, qtype_name, rcode_name
```

20 events (8/15/26 12:36:16.000 PM to 8/15/26 12:51:16.000 PM) No Event Sampling

Events

Patterns

Statistics (20)

Visualization

Show: 20 Per Page Format Preview: On

_time	src_ip	dest_ip	query	qtype_name	rcode_name
2026-08-15 12:43:18.434	192.168.23.131	192.168.23.2	netflix.com	AAAA	NOERROR
2026-08-15 12:43:18.370	192.168.23.131	192.168.23.2	netflix.com	A	NOERROR
2026-08-15 12:43:17.194	192.168.23.131	192.168.23.2	amazonaws.com	AAAA	NOERROR
2026-08-15 12:43:17.129	192.168.23.131	192.168.23.2	amazonaws.com	A	NOERROR
2026-08-15 12:43:15.935	192.168.23.131	192.168.23.2	microsoft.com	AAAA	NOERROR
2026-08-15 12:43:15.762	192.168.23.131	192.168.23.2	microsoft.com	A	NOERROR
2026-08-15 12:43:14.633	192.168.23.131	192.168.23.2	wikipedia.org	AAAA	NOERROR
2026-08-15 12:43:14.563	192.168.23.131	192.168.23.2	wikipedia.org	A	NOERROR

Query, source IP, destination IP, DNS record type, and response code — all successfully pulled out of the raw Zeek data, without relying on the add-on at all.

localhost:8000/en-US/app/search/search?q=search%20index%3Dmain%20sourcetype%3Dzeek%20source%3D*dns.log&display.page.search.mode=smart&dispatch.s...

splunk> Enterprise

New Search

Save As

Create Table View

Close

Time range: Last 24 hours

88 events (8/14/26 10:00:00.000 AM to 8/15/26 10:52:33.000 AM) No Event Sampling

Events (88)

Patterns

Statistics

Visualization

Timeline format Zoom Out Zoom to Selection Deselect 1 hour per color

Format Show: 20 Per Page View: List

< Hide Fields

All Fields

SELECTED FIELDS

a host 1

a source 1

a sourcetype 1

INTERESTING FIELDS

date_hour 2

date_mday 2

date_minute 10

date_month 1

date_second 34

date_wday 2

date_year 1

i	Time	Event
>	8/15/26 10:47:25.933 AM	1786787245.933333 CcQnQc49MhSIUXXs2 192.168.23.131 59640 192.168.23.2 53 udp 39446 - c2-beacon-tt -10.doesnotexist-fake.com 1 C_INTERNET 1 A 3 NXDOMAIN F F T F 0 host = kali source = /home/kali/dns.log sourcetype = zeek
>	8/15/26 10:47:23.578 AM	1786787243.578647 C8whbk3eFH1MV0LW2h 192.168.23.131 48215 192.168.23.2 53 udp 40144 - c2-beacon-tt -9.doesnotexist-fake.com 1 C_INTERNET 1 A 3 NXDOMAIN F F T F 0 host = kali source = /home/kali/dns.log sourcetype = zeek
>	8/15/26 10:47:20.797 AM	1786787240.797924 CX60dKtdaHnpBGjM 192.168.23.131 41475 192.168.23.2 53 udp 39430 - c2-beacon-tt -8.doesnotexist-fake.com 1 C_INTERNET 1 A 3 NXDOMAIN F F T F 0 host = kali source = /home/kali/dns.log sourcetype = zeek

Confirming Zeek's live DNS data was actually landing in Splunk before I built any detection logic on top of it.

What I Found

Quick reference before the details — how each finding maps to the **MITRE ATT&CK framework**, the industry-standard catalog of real-world attacker tactics and techniques used across the security industry:

Finding	Tactic	Technique
DNS Tunneling	Exfiltration (TA0010)	T1048.003 — Exfiltration Over Unencrypted/Obfuscated Non-C2 Protocol
C2 Beaconsing	Command and Control (TA0011)	T1071.004 — Application Layer Protocol: DNS

Finding 1 — DNS Tunneling Indicators

To hunt for tunneling, I needed a way to flag subdomains that look machine-generated instead of human-typed. Two things tend to give this away: **length** (a human doesn't type a 16-character random subdomain) and **digit ratio** — what percentage of the characters are numbers, since real words have very few digits and random hex strings have a lot. My search calculates both for every subdomain, then filters for anything longer than 12 characters, well beyond what normal domains use.

The screenshot shows the Splunk Enterprise search interface. The search bar contains the following query:

```
index=main host=kali source="*dns.log"
| eval fields=split(_raw, " ")
| eval ts=mvindex(fields,0), src_ip=mvindex(fields,2), query=mvindex(fields,9)
| where NOT match(_raw, "#") AND isnotnull(query) AND query!="
| eval subdomain=mvindex(split(query, "."), 0)
| eval sub_len=len(subdomain)
| eval digit_count=len(replace(subdomain, "[^0-9]", ""))
| eval digit_ratio=round(digit_count/sub_len, 2)
| stats count avg(sub_len) as avg_length, max(sub_len) as max_length, avg(digit_ratio) as avg_digit_ratio by query
| where max_length > 12
| sort -max_length
```

The search results show 41 events. The table below displays the first six results, which are all from the domain `exfil-test-domain.com`.

query	count	avg_length	max_length	avg_digit_ratio
0ef2ff70da7b3b5.exfil-test-domain.com	1	16	16	
2b65d538e2930063.exfil-test-domain.com	1	16	16	
2e871dc891667cce.exfil-test-domain.com	1	16	16	
3763e50c549ab7e7.exfil-test-domain.com	1	16	16	
46788a82f7161b83.exfil-test-domain.com	1	16	16	
4c7e64a9ed81066.exfil-test-domain.com	1	16	16	

41 matching events, with 26 distinct subdomains flagged as unusually long.

What I saw

Over 40 queries went to 16-character random subdomains under one domain, `exfil-test-domain.com`, with digit ratios between 0.44 and 0.81 (meaning up to 81% of the characters were digits). Compare that to my

baseline domains like google.com — a digit ratio of 0, since there isn't a single number in it.

Why it matters

This length-and-randomness pattern is a real, documented signature of DNS tunneling. Attackers use it because DNS is rarely blocked outbound, so hiding data inside a DNS query is a reliable way to sneak it past a firewall that's watching everything else.

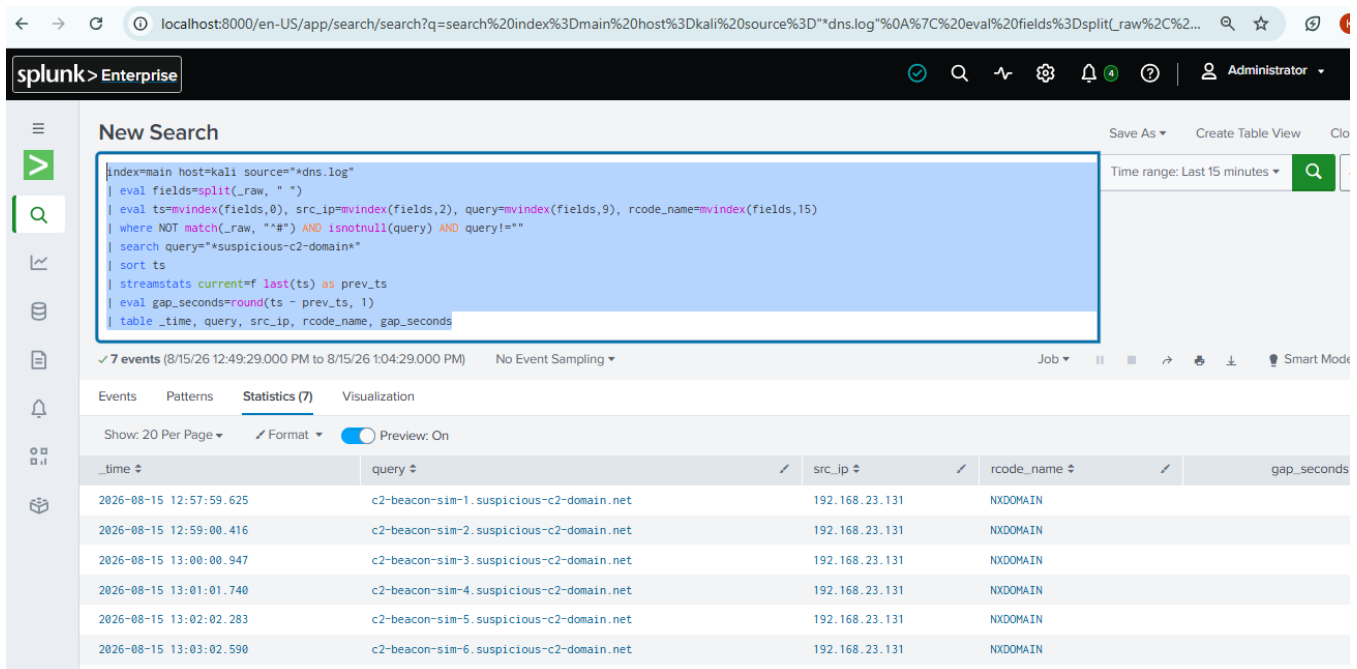
What I'd do next

In a real environment, this pattern under a single host would be enough to justify isolating that machine and checking it for exfiltration tools or signs of compromise — before more data goes out.

MITRE ATT&CK MAPPING	
Tactic	Exfiltration (TA0010)
Technique	T1048 — Exfiltration Over Alternative Protocol
Sub-technique	T1048.003 — Exfiltration Over Unencrypted/Obfuscated Non-C2 Protocol
Related	T1071.004 — Application Layer Protocol: DNS, if the same channel is also used for command-and-control rather than exfiltration alone

Finding 2 — C2 Beaconsing Pattern

For beaconsing, timing is the tell. I sorted my suspicious queries by time and calculated the gap between each one and the query right before it, using Splunk's streamstats command.



```
index=main host=kali source="*dns.log"
| eval fields=split(_raw, " ")
| eval ts=mindex(fields,0), src_ip=mindex(fields,2), query=mindex(fields,9), rcode_name=mindex(fields,15)
| where NOT match(_raw, "^#") AND isnotnull(query) AND query!=" "
| search query="*suspicious-c2-domain*"
| sort ts
| streamstats current=f last(ts) as prev_ts
| eval gap_seconds=round(ts - prev_ts, 1)
| table _time, query, src_ip, rcode_name, gap_seconds
```

_time	query	src_ip	rcode_name	gap_seconds
2026-08-15 12:57:59.625	c2-beacon-sin-1.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	
2026-08-15 12:59:00.416	c2-beacon-sin-2.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	
2026-08-15 13:00:00.947	c2-beacon-sin-3.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	
2026-08-15 13:01:01.740	c2-beacon-sin-4.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	
2026-08-15 13:02:02.283	c2-beacon-sin-5.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	
2026-08-15 13:03:02.590	c2-beacon-sin-6.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	

13 queries to suspicious-c2-domain.net, spaced 60.3–60.9 seconds apart, every single one coming back NXDOMAIN (the domain doesn't exist).

What I saw

The queries landed almost exactly 60 seconds apart, every time — no human browsing session naturally produces that kind of mechanical regularity. On top of that, every single query failed to resolve (NXDOMAIN).

Why it matters

Malware often “phones home” to its C2 server on a fixed timer, checking for new instructions. A domain that fails to resolve 100% of the time, combined with clockwork-regular timing, is a well-known combination analysts look for — it suggests the C2 server is either inactive, taken down, or not yet set up, but the malware is still trying.

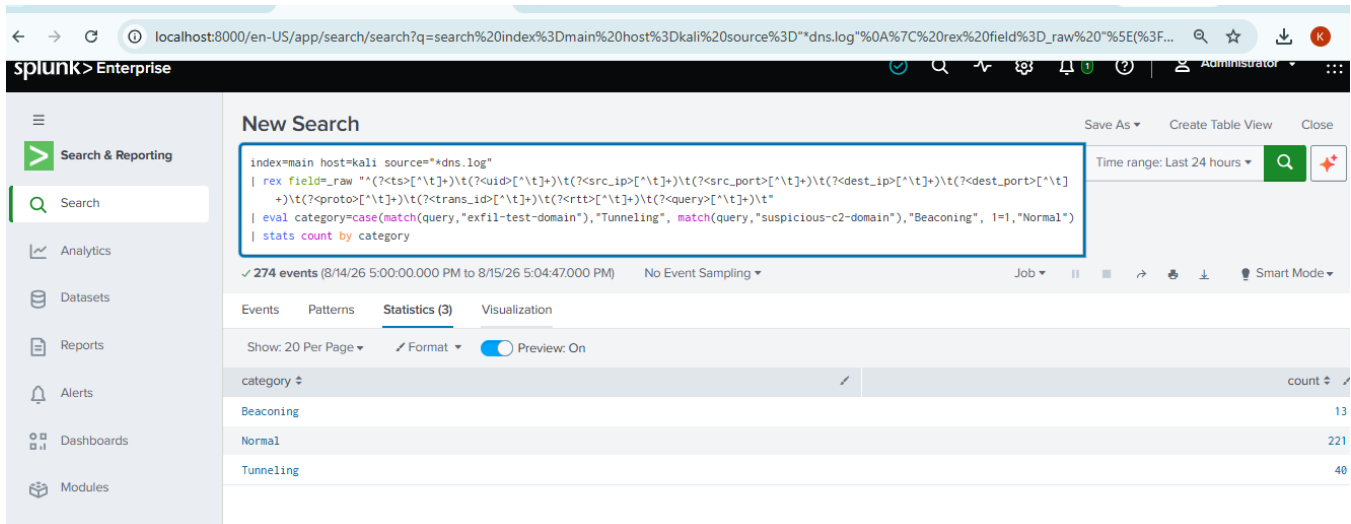
What I'd do next

This combination — fixed interval, total resolution failure — would be enough for me to flag the source host for deeper endpoint investigation, checking for actual malware rather than assuming it's a fluke.

MITRE ATT&CK MAPPING	
Tactic	Command and Control (TA0011)
Technique	T1071.004 — Application Layer Protocol: DNS (using DNS as the covert channel for C2 check-ins)
Related	T1568 — Dynamic Resolution, worth investigating further in case the beacon domains were being algorithmically generated rather than static

Finding 3 — Separating Normal Traffic from the Two Attack Simulations

Finding suspicious traffic is only half the job — I also wanted to prove I could tell it apart from everything else in the same dataset, not just search for it in isolation. This search sorts every DNS query into one of three buckets (Normal, Tunneling, Beaconsing) using simple pattern matching on the domain name.



The screenshot shows the Splunk Enterprise search interface. The search bar contains the following query:

```
index=main host=kali source="*dns.log"
| rex field=_raw "(?<ts>[^\t]+\t(?<uid>[^\t]+\t(?<src_ip>[^\t]+\t(?<src_port>[^\t]+\t(?<dest_ip>[^\t]+\t(?<dest_port>[^\t]+)
+)\t(?<proto>[^\t]+\t(?<trans_id>[^\t]+\t(?<rtt>[^\t]+\t(?<query>[^\t]+\t"
| eval category=case(match(query,"exfil-test-domain"),"Tunneling", match(query,"suspicious-c2-domain"),"Beaconing", 1="Normal")
| stats count by category
```

The search results show 274 events. The results are displayed in a table with the following columns: category and count.

category	count
Beaconing	13
Normal	221
Tunneling	40

274 total DNS events, split into 221 Normal, 40 Tunneling, and 13 Beaconing.

What I saw

Out of 274 total DNS events I captured, the search correctly separated them: 221 normal, 40 matching the tunneling pattern, 13 matching the beaconing pattern.

Why it matters

In a real environment, you're never handed "suspicious traffic" pre-filtered — you have to pull it out of everything else yourself. Being able to quantify how much of your traffic is normal versus suspicious is usually the first step in triaging what's actually going on.

I also ran a follow-up search to check exactly where the tunneling traffic came from and when:

localhost:8000/en-US/app/search/search?q=search%20index%3Dmain%20host%3Dkali%20source%3D*dns.log%0A%7C%20rex%20field%3D_raw%20*%5E(%3F...

splunk> Enterprise

New Search

```
index=main host=kali source="*dns.log"
| rex field=_raw "(?<ts>[^\t]+\t(?<uid>[^\t]+\t(?<src_ip>[^\t]+\t(?<src_port>[^\t]+\t(?<dest_ip>[^\t]+\t(?<dest_port>[^\t]+)\t(?<proto>[^\t]+\t(?<trans_id>[^\t]+\t(?<rtt>[^\t]+\t(?<query>[^\t]+\t"
| where match(query, "exfil-test-domain")
| stats count values(src_ip) as source_ips, values(dest_ip) as dest_ips, earliest(_time) as first_seen, latest(_time) as last_seen
by query
```

Time range: Last 24 hours

✓ 40 events (8/14/26 5:00:00.000 PM to 8/15/26 5:01:36.000 PM) No Event Sampling

Events Patterns **Statistics (40)** Visualization

Show: 20 Per Page Format Preview: On

query	count	source_ips	dest_ips	first_seen	last_seen
0ef2ff70d1a7b3b5.exfil-test-domain.com	1	192.168.23.131	192.168.23.2	1786795016.366579	1786795016.366579
133aea56ec01d820.exfil-test-domain.com	1	192.168.23.131	192.168.23.2	1786787176.789889	1786787176.789889
16e9247bda869620.exfil-test-domain.com	1	192.168.23.131	192.168.23.2	1786787179.320081	1786787179.320081
29fbeb90e1f15443.exfil-test-domain.com	1	192.168.23.131	192.168.23.2	1786787175.120842	1786787175.120842
2ab5e1c156a4c70a.exfil-test-domain.com	1	192.168.23.131	192.168.23.2	1786787183.024405	1786787183.024405
2b65d538e2930063.exfil-test-domain.com	1	192.168.23.131	192.168.23.2	1786795026.384403	1786795026.384403
2e871dc891667cce.exfil-test-domain.com	1	192.168.23.131	192.168.23.2	1786795011.368803	1786795011.368803
3763e50c549ab7e7.exfil-test-domain.com	1	192.168.23.131	192.168.23.2	1786794999.364512	1786794999.364512

All 40 tunneling-pattern queries traced back to a single source host (192.168.23.131), within a tightly bounded time window.

What I'd do next

Since every tunneling-pattern query came from one host, in a tight time window, that's a strong, specific finding — not a vague “something might be wrong somewhere.” That specificity is exactly what makes a finding actionable for escalation.

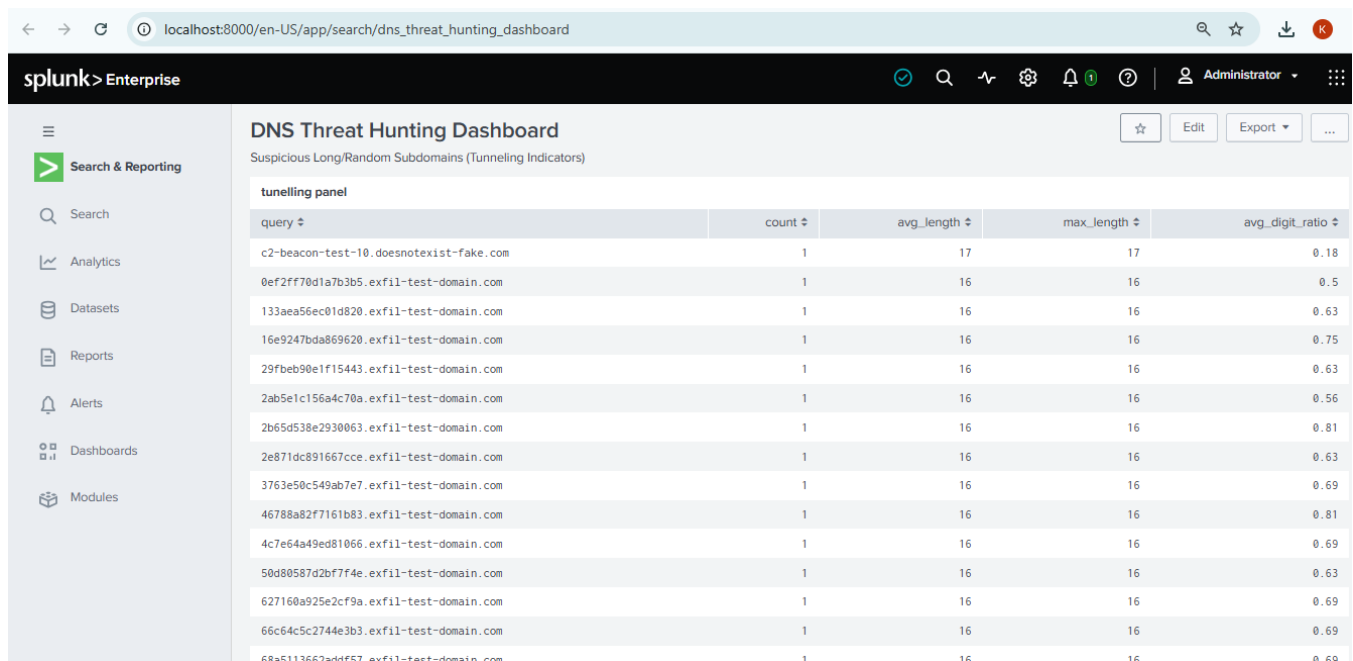
MITRE ATT&CK MAPPING

Tactic	N/A — detection methodology, not an attack technique
Technique	Data Source: DS0035 — DNS (query logs)
Related	This is the data source ATT&CK lists as required to detect both T1048.003 and T1071.004 above — exactly what Zeek's dns.log provided here.

The Dashboard

I brought all three findings together into one Splunk dashboard, **DNS Threat Hunting Dashboard**, with three panels:

- Suspicious Long/Random Subdomains (Tunneling Indicators) — a statistics table
- C2 Beaconsing Pattern Detection (Regular Interval NXDOMAIN Queries) — a table showing the calculated time gaps
- Baseline DNS Query Volume (Normal Traffic) — a time chart, so the suspicious findings have visual “normal” traffic to stand out against



The screenshot shows the Splunk Enterprise interface with the 'DNS Threat Hunting Dashboard' open. The dashboard has a sidebar with navigation options: Search & Reporting, Search, Analytics, Datasets, Reports, Alerts, Dashboards, and Modules. The main content area displays the 'tunneling panel' with a table of suspicious subdomains. The table has five columns: query, count, avg_length, max_length, and avg_digit_ratio. The data is sorted by query. The table contains 15 rows of data, each representing a different subdomain and its associated metrics.

query	count	avg_length	max_length	avg_digit_ratio
c2-beacon-test-10.doesnotexist-fake.com	1	17	17	0.18
0ef2ff70d1a7b3b5.exfil-test-domain.com	1	16	16	0.5
133aea56ec01d820.exfil-test-domain.com	1	16	16	0.63
16e9247bda869620.exfil-test-domain.com	1	16	16	0.75
29fbeb90e1f15443.exfil-test-domain.com	1	16	16	0.63
2ab5e1c156a4c70a.exfil-test-domain.com	1	16	16	0.56
2b65d538e2930063.exfil-test-domain.com	1	16	16	0.81
2e871dc891667cce.exfil-test-domain.com	1	16	16	0.63
3763e50c549ab7e7.exfil-test-domain.com	1	16	16	0.69
46788a82f7161b83.exfil-test-domain.com	1	16	16	0.81
4c7e64a49ed81066.exfil-test-domain.com	1	16	16	0.69
50d80587d2bf7f4e.exfil-test-domain.com	1	16	16	0.63
627160a925e2cf9a.exfil-test-domain.com	1	16	16	0.69
66c64c5c2744e3b3.exfil-test-domain.com	1	16	16	0.69
68a5113662addf57.exfil-test-domain.com	1	16	16	0.69

The tunneling panel, showing every flagged subdomain with its length and digit ratio.

localhost:8000/en-US/app/search/dns_threat_hunting_dashboard

C2 Beaconsing Pattern Detection (Regular Interval NXDOMAIN Queries)

_time ↕	query ↕	src_ip ↕	rcode_name ↕	gap_seconds ↕
2026-08-15 12:57:59.625	c2-beacon-sim-1.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	
2026-08-15 12:59:00.416	c2-beacon-sim-2.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	60.8
2026-08-15 13:00:00.947	c2-beacon-sim-3.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	60.5
2026-08-15 13:01:01.740	c2-beacon-sim-4.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	60.8
2026-08-15 13:02:02.283	c2-beacon-sim-5.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	60.5
2026-08-15 13:03:02.590	c2-beacon-sim-6.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	60.3
2026-08-15 13:04:03.308	c2-beacon-sim-7.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	60.7
2026-08-15 13:05:03.923	c2-beacon-sim-8.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	60.6
2026-08-15 13:06:04.860	c2-beacon-sim-9.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	60.9
2026-08-15 13:07:05.297	c2-beacon-sim-10.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	60.4
2026-08-15 13:08:05.638	c2-beacon-sim-11.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	60.3
2026-08-15 13:09:06.039	c2-beacon-sim-12.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	60.4
2026-08-15 13:10:06.575	c2-beacon-sim-13.suspicious-c2-domain.net	192.168.23.131	NXDOMAIN	60.5

Baseline DNS Query Volume=

_time ↕	VALUE_dosvc_tcp.local ↕	VALUE_googlecast_tcp.local ↕	amazonaws.com ↕	desktop-16b94ok ↕	f.c.a.e.870.9f95.0.e8.6.3.0.0.0.0.0.0.0.0.8.e.f.ip6.arpa ↕	github ↕
2026-08-14 17:00:00	0	0	0	0	0	0
2026-08-14 17:01:00	0	0	0	0	0	0
2026-08-14 17:02:00	0	0	0	0	0	0

The beaconsing panel (top), showing the ~60-second gaps between each suspicious query, and the baseline query volume panel (bottom) for comparison against normal traffic.

Skills I Practiced in This Project

- Capturing and forwarding live network traffic through a real pipeline (Zeek, Splunk Universal Forwarder)
- Writing Splunk SPL: rex, eval, stats, streamstats, timechart, case()
- Manually extracting fields with regex when a vendor add-on's parsing failed
- Understanding and detecting two real DNS-based attack techniques: tunneling and C2 beaconsing
- Mapping findings to the MITRE ATT&CK framework
- Building a SOC-style dashboard for reporting findings visually
- Writing up an investigation the way an analyst actually documents one: observation → why it matters → next action

What's Next

I'm reusing this same pipeline and approach for my next project: **SSH Brute-Force Detection** — running a real brute-force attempt from Kali against a lab target, and building Splunk detections against ssh.log.

Appendix: SPL Queries

Full search syntax for each detection, provided for technical reference. Runnable copies are also included in the project's GitHub repository under `splunk_searches/`.

Tunneling Detection

```
index=main host=kali source="*dns.log"
| rex field=_raw "^(?<ts>[^\t]+\t(?<uid>[^\t]+\t(?<src_ip>[^\t]+\t(?<src_port>[^\t]+\t(?<dest_ip>[^\t]+\t(?<dest_port>[^\t]+\t(?<proto>[^\t]+\t(?<trans_id>[^\t]+\t(?<rtt>[^\t]+\t(?<query>[^\t]+\t"
| where isnotnull(query) AND query!="
| eval subdomain=mvindex(split(query, "."), 0)
| eval sub_len=len(subdomain)
| eval digit_count=len(replace(subdomain, "[^0-9]", ""))
| eval digit_ratio=round(digit_count/sub_len, 2)
| stats count avg(sub_len) as avg_length, max(sub_len) as max_length, avg(digit_ratio) as avg_digit_ratio by query
| where max_length > 12
| sort -max_length
```

C2 Beaconsing Detection

```
index=main host=kali source="*dns.log"
| rex field=_raw "^(?<ts>[^\t]+\t...(?<query>[^\t]+\t...(?<rcode_name>[^\t]+)"
| search query="*suspicious-c2-domain*"
| sort ts
| streamstats current=f last(ts) as prev_ts
| eval gap_seconds=round(ts - prev_ts, 1)
| table _time, query, src_ip, rcode_name, gap_seconds
```

Traffic Classification

```
index=main host=kali source="*dns.log"
| rex field=_raw "^(?<ts>[^\t]+\t...(?<query>[^\t]+\t"
| eval category=case(match(query, "exfil-test-domain"), "Tunneling",
match(query, "suspicious-c2-domain"), "Beaconing", 1=1, "Normal")
| stats count by category
```